

Biometric Steering Wheel System Guide

Every part of the working system — sensor, ESP32 firmware, signal processing, Raspberry Pi relay, phone app, warning and emergency system, cloud database and live website — with setup instructions for any Linux machine, test results and references.



Branch
luis/full-system

Live dashboard
smart-wheel-dashboard.vercel.app

Date
September 22, 2026

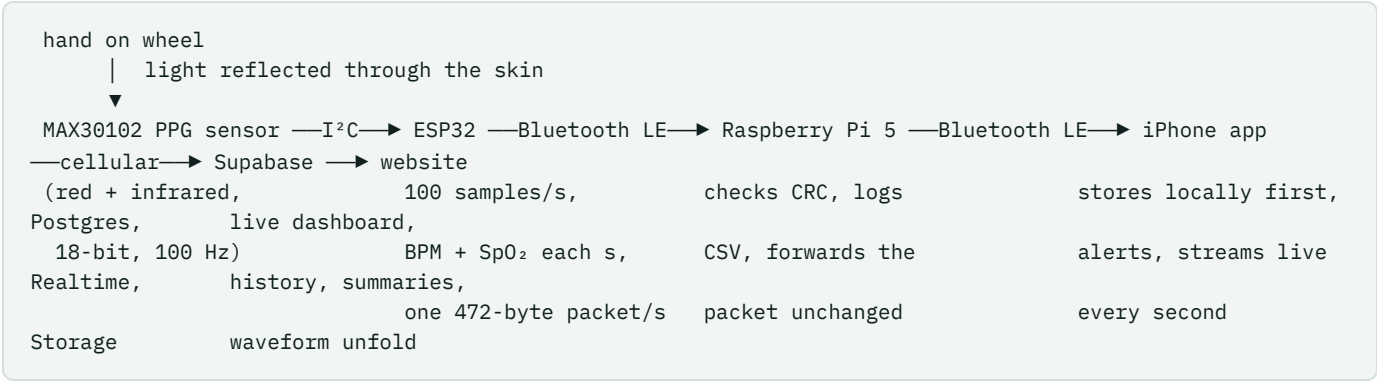
Source of this document
docs/SYSTEM_GUIDE.md

Contents

01	The system in one picture
02	Hardware: sensor and ESP32
03	ESP32 firmware
04	Signal processing and algorithms
05	The radio packet (protocol v3)
06	Raspberry Pi relay
07	Phone app
08	Cloud: Supabase
09	Website dashboard
10	Data policy, storage and backups
11	Warning and emergency system
12	11.1 What is "normal" for this driver (the profile)
13	11.2 Levels
14	11.3 Warning → emergency ("watch the next seconds")
15	11.4 The voice check (phone only, free, offline)
16	11.5 Irregular-rhythm advisory
17	11.6 What gets recorded
18	11.7 Accuracy and learning
19	11.8 Watching it adapt, and tuning it from the team's data
20	Setting up a development machine (any Linux)
21	Everyday commands
22	Tests
23	Measured results
24	Troubleshooting
25	Security status and before-real-data checklist

26	How this maps to the proposal
27	Scaling the wheel: 12-16 sensor pads
28	Electrical
29	Firmware
30	Packet
31	What each pad count buys
32	Drawing it
33	References

1. The system in one picture



PART	RUNS	WHERE THE CODE IS
Sensor + ESP32	Arduino C++	system/esp32/ppg_transmitter/
Pi relay + terminal dashboard	Python 3 (asyncio, bleak, dbus-fast, rich)	system/pi/
Laptop viewer (bench tool)	Python 3 (PySide6, pyqtgraph)	system/laptop/
Shared protocol code	Python	system/common/ (copied into pi/ and laptop/)
Phone app	React Native + Expo SDK 54, TypeScript	mobile-app/
Database	Supabase (Postgres 17, Realtime, Storage)	mobile-app/supabase/*.sql
Website	Vite + vanilla JS, uPlot, fflate	website/

Key design ideas

- **Only the phone needs the internet.** In the car the ESP32 and the Pi talk only over Bluetooth. The phone uses its cellular data to reach Supabase.
- **Nothing is lost when there is no signal.** The phone saves every reading locally first, then uploads. Uploads are retried, and the same row can be sent twice without creating a duplicate.
- **Errors are caught end to end.** The ESP32 adds a CRC-16 checksum to every packet. The Pi forwards packets byte-for-byte, and the phone checks the ESP32's checksum again. Corruption anywhere on the path is detected.

2. Hardware: sensor and ESP32

ITEM	DETAIL
Microcontroller	ESP32-D0WD-V3 (rev 3.1) dev board, CP210x USB-serial (USB ID <code>10c4:ea60</code>)
Sensor	MAX30102 pulse-oximetry module, I ² C address <code>0x57</code> (PART_ID reads <code>0x15</code>)
Wiring	module 3V3 → ESP32 3V3 , GND → GND , SDA → GPIO 21 , SCL → GPIO 22
Pi	Raspberry Pi 5, Raspberry Pi OS, Bluetooth + Wi-Fi from its onboard Infineon CYW43455

Wiring notes from the bench:

- SCL must be on **GPIO 22**. On GPIO 23 the sensor is not found. The firmware prints I²C diagnostics if the sensor is missing: which addresses answer, and whether the module's pull-up resistors are visible.
- **Car power.** The ESP32 runs from the car's 12 V through a 12 V → 5 V USB adapter. The Pi should get an ignition-switched supply. Sudden power loss can corrupt the Pi's SD card, so a UPS HAT or a supercapacitor plus an ignition-sense line is recommended before real use.

3. ESP32 firmware

File: `system/esp32/ppg_transmitter/ppg_transmitter.ino`, with `ppg_frame.h` (packet encoder) and `ppg_vitals.h` (heart rate and SpO₂).

What it does every second:

1. **Reads the sensor FIFO directly** (registers `0x04 – 0x07`). The sensor samples at **100 Hz**: SpO₂ mode, 411 μ s LED pulse, 18-bit ADC, 4096 nA range. The datasheet allows 50/100/200/400 sps at 411 μ s. SparkFun's `getRed()/getIR()` are not used. Measured on this board, they block and return the *newest* sample, which ran acquisition 2.5 $\times$ slow and paired red and IR values from different moments.
2. **Keeps all 100 raw samples** for the packet.
3. **Averages groups of 4 samples into a 25 Hz series** (decimation) for the vitals estimator and keeps an 8-second history.
4. **Estimates heart rate and SpO₂** (see §4). The finger check uses the infrared (IR) DC level, which must be at least **100 000**.
5. **Builds a v3 packet** (§5): 472 bytes, CRC-16. It sends the packet in pieces sized to the Bluetooth link, because the stack silently truncates anything larger than MTU–3.
6. **Prints one line to serial**, for example: `#105 bpm=72 spo2=99 q=0.83 R=0.439 maxim=136/99 ir=256361 finger=1 win=999ms ovf=0 link=up mtu=247 len=472 conns=3 lastdisc=0x13 crc=3771` (`maxim=` is the old reference algorithm, printed only for comparison; `ovf` counts sensor samples dropped because we fell behind, and should be 0; `conns / lastdisc` show how many times the Pi connected and why the last link ended, as an HCI reason code).

Bluetooth settings:

- Advertises as `SteeringWheelESP32` with the Nordic UART Service UUIDs.
- Transmit power is **+9 dBm**, the maximum. The default was +3 dBm.
- Asks the Pi for a connection interval of 30–50 ms, latency 0 and a supervision timeout of 4 s.

Build settings: flash at **40 MHz DIO**. At the default 80 MHz QIO this board failed its boot checksum and rebooted in a loop.

4. Signal processing and algorithms

Filtering chain (on the ESP32)

STAGE	WHAT	WHY
On-chip	18-bit sigma-delta ADC, 411 μ s pulses, ambient-light cancellation in the MAX30102	clean raw optical signal
Decimation	mean of each 4 consecutive 100 Hz samples $\rightarrow$ 25 Hz	box-car low-pass; cuts high-frequency noise, and 25 Hz is plenty for heart rate
Baseline removal	subtract a centred 1-second moving average	removes DC and slow drift (breathing, pressure changes)
Smoothing	3-tap filter, -3 dB near 8 Hz at 25 Hz	smooths noise but keeps the pulse harmonics
Finger gate	IR DC $\geq 100\,000$ counts	measured: finger $\approx 242\,000$, object resting $\approx 50\,400$, open air $\approx 1\,100$
Plausibility	BPM 30–220, SpO ₂ 70–100	anything outside is flagged as not usable

Heart rate: autocorrelation (`ppg_vitals.h`)

- Works on 8 s of the pulsatile infrared signal at 25 Hz.
- Computes the normalised autocorrelation and takes the first peak that reaches **0.5** or more as the beat period.
- Refines that peak with parabolic interpolation, giving sub-sample precision.
- Heart rate = 60 / period.
- The peak's height (0–1) is the **quality** score sent in every packet.

It replaced Maxim's reference algorithm. On this hardware, Maxim's median was 115 BPM when independent estimates said 79–80. It counts the diastolic notch as an extra beat. The new estimator's median was 65 BPM with a standard deviation of 3.3.

SpO₂: ratio of ratios

- $R = (AC_{red} / DC_{red}) / (AC_{ir} / DC_{ir})$, where AC is the RMS over the window.
- R is looked up in Maxim's own calibration table (`uch_spo2_table`), so the calibration is the manufacturer's.

On the Pi: a 5-point moving average of usable readings, for the display only.

On the phone and website, summaries use percentiles, not min/max. PPG on a steering wheel picks up grip and motion artifacts, and a single bad second would otherwise become the "highest heart rate" of a drive.

- **Realistic low** = 5th percentile, **typical** = median, **realistic high** = 95th percentile. Min and max are still shown in small print.
- The phone and the database use the same percentile definition (linear interpolation, like `Postgres percentile_cont`), so they report identical numbers. This is unit-tested.

Why 100 Hz raw data matters. Mean heart rate needs only a low sampling rate. Pulse-rate variability is reliable at 25 Hz or more in healthy subjects (Choi & Shin 2017). RMSSD and frequency-domain HRV need about 100–200 Hz unless you interpolate (Béres et al. 2019). Sending every 100 Hz sample keeps those analyses possible for the team's future algorithm work.

5. The radio packet (protocol v3)

One packet per second. Little-endian. The CRC is computed by the ESP32 and checked by the Pi, the phone and the laptop.

OFFSET	SIZE	FIELD	NOTES
0	2	magic <code>0xA55A</code>	on the wire <code>5A A5</code>
2	1	version	3
3	1	flags	1 = HR valid, 2 = SpO ₂ valid, 4 = finger, 8 = in range
4	4	seq	+1 per second, restarts at 0 when the ESP32 reboots
8	4	t0_ms	ESP32 clock at the first sample
12	2	rate_hz	100
14	1	n	samples in this packet (100)
15	1	quality	0–100
16	2	heart_rate	BPM, –999 = no result
18	2	spo2	%, –999 = no result
20	[36n/8]	samples	n × {red:18 bits, IR:18 bits}, bit-packed
end	2	CRC-16/CCITT-FALSE	check value <code>0x29B1</code>

- Sample i was taken at `t0_ms + i × 1000 / rate_hz`, on the sensor's own clock.
- v3 carries **12.5× the samples of v2 for 4.5× the bytes**. v2 sent 8 of the 100 samples in 104 bytes. The 18 bits match the sensor's ADC exactly, so no data is lost.
- The receivers also decode v2, so old captures still read.
- **The deframer** reassembles packets from Bluetooth pieces of any size. It resynchronises on the magic bytes after corruption. If it has buffered a false header, it looks ahead for a complete valid packet instead of waiting on the false one.
- **Implementations are kept in lock-step by tests:**
 - C++ `ppg_frame.h` (encoder)
 - Python `system/common/ppg_protocol.py`
 - TypeScript `mobile-app/lib/ble/protocol.ts`

Both the Python and TypeScript test suites decode packets produced by the compiled firmware encoder.

Bluetooth IDs

	SERVICE	CHARACTERISTIC
ESP32	6e400001-b5a3-f393-e0a9-e50e24dcca9e	TX notify 6e400003-...
Pi relay (PPG-Relay - Pi)	5b1e0001-7c2d-4f6a-9e3b-8a4c2d1f6e01	frames 5b1e0002-... (notify), status JSON 5b1e0003-... (read)

6. Raspberry Pi relay

Files (in `system/pi/` , installed to `~/PPG_Logger` on the Pi):

- `ppg_relay.py` — one asyncio process that:
 - connects to the ESP32 as a Bluetooth central (bleak) and verifies every packet;
 - logs processed values to `~/PPG_Logger/logs/ppg_data.csv` (owner-only permissions, umask 077). Columns: timestamp, spo2, avg_spo2, bpm, avg_bpm, seq, esp times, validity flags, quality, protocol, samples, rate_hz;
 - runs a GATT server and advertisement for the phone (dbus-fast → BlueZ), forwarding each packet unchanged in MTU-sized pieces;
 - writes live state to `/run/ppg-relay/state.json` for the dashboard;
 - retries the phone-side service every 5 s if Bluetooth is off at boot;
 - backs off between failed ESP32 attempts, from 1 s up to 15 s;
 - releases BlueZ links orphaned by a crash, and closes a dead BlueZ link to the ESP32 after any failed connect (seen after a service restart: BlueZ said "Connected: yes" while the ESP32 was advertising, so every connect timed out);
 - ignores a stale disconnect event that BlueZ can emit during the connect handshake. bleak 3 passes it to `disconnected_callback` although the link is up; acting on it made the relay hang up (HCI 0x13) about 3 s after every connect. The link now counts as lost only when `client.is_connected` agrees.
- `ppg_monitor.py` — the terminal dashboard. It opens automatically at boot and shows the links, vitals with sparklines, latest packets, counters (including the packet format and bytes/s) and events. `ppg-monitor --once` prints one frame.
- `setup_pi.sh` — one-shot installer (`sudo bash setup_pi.sh`). It:
 - installs bluez and python3-venv;
 - unblocks and powers Bluetooth;
 - writes the [LE] connection and scan parameters to `/etc/bluetooth/main.conf`;
 - builds a venv from the bundled wheels in `pi/wheels/` , so no internet is needed;
 - installs the systemd service `ppg-relay` (`Restart=always` , `UMask=0077`);
 - sets the dashboard to autostart.

Options: `--no-service` , `--no-dashboard` , `--uninstall` .

- **Raw samples are off by default.** `--log-raw` stores them for development only.

Important: the Pi 5's Wi-Fi and Bluetooth share one radio and antenna. Measured on the bench:

- While the Pi's Wi-Fi was on 2.4 GHz, the ESP32 link failed to establish (HCI reason 0x3E) again and again.
- Switching Bluetooth on made Wi-Fi glitch.
- **With Wi-Fi off, the link held steadily** (after the relay fix below).

- Rebooted with Wi-Fi on (2.4 GHz, 40 % signal): not one connection request reached the ESP32 in several minutes, although the Pi's scans found it and a laptop connected fine.

The "connected → scanning → connected" loop seen on the bench had a separate cause: the relay acted on a stale BlueZ disconnect event and closed every link itself after ~3 s (ESP32 logged `lastdisc=0x13`, central terminated). Fixed in `ppg_relay.py`; the laptop and the Pi then held the link for minutes. How to read the ESP32's `lastdisc`: `0x13` = the Pi hung up (look at the relay), `0x08` = the Pi went silent (radio/coexistence/range), `0x3E` = the connection never got established (radio/coexistence/range).

What to do about it:

- **In the car:** keep Wi-Fi off (`sudo nmcli radio wifi off`). Nothing on the Pi needs it.
- **On the bench:** the relay now scans at a 25% duty cycle and backs off after failures. It is still best to use Ethernet, or to keep Wi-Fi off while testing Bluetooth.
- **Placement:** keep the ESP32 within about 1–2 m of the Pi, and never with a laptop or other metal between them. The Pi heard the ESP32 at –93 dBm with a laptop in between, which is at the limit of sensitivity.

7. Phone app

React Native (Expo SDK 54), TypeScript, in `mobile-app/`.

Screens

- **Introduction** (first launch only; reopen from Settings → Your data): five short pages covering what it does, how the check works, the profile, your data, and voice setup (microphone permission and voice choice).
- **Who's driving?** — pick, add or edit a driver: name, subject ID, age, weight, height (the app shows **BMI** live), gender, health conditions, medications, voice language and an emergency contact. To edit, long-press a driver. Once in, **Switch driver** in the header goes back; if a drive is recording, it is ended and saved first.
- **Drive**
 - Phone → Pi → Wheel connection chain.
 - Big heart-rate and oxygen tiles.
 - 2-minute charts, with the driver's usual range shaded.
 - Start/End drive.
 - A **Data flow** card with a live dot that moves with the bytes/s coming from the wheel, plus the bytes/s sent to the cloud.
- **History** — past drives with realistic low / typical / realistic high, alerts, and **Unfold waveform** for full-waveform drives. Unfold checks the SHA-256, decompresses, and shows 10 s of red and IR.
- **Settings**
 - What to save (§10).
 - **Safety checks:** this driver's thresholds, how the profile set them, the voice-check status, and **Try the voice check** (§11).
 - Upload waiting drives.
 - Auto-connect on/off.
 - Link health counters.
 - Bench-only "connect straight to ESP32".
 - **Your data:** show the introduction again, and **wipe all data on this phone**.
- **Deleting.**
 - A driver: long-press them, then Delete.
 - One drive: History → open it → Delete.
 - Everything: Settings → Wipe.

Each deletes on the phone **and** in Supabase: the driver row cascades to their drives, readings, alerts and archive records, and the waveform files are removed from Storage. Offline deletions are queued and retried at start and on every sync. Only data this phone recorded is touched, never other phones' data. Verified against the live database: the test driver, drive and reading were all gone after one delete.

How it connects: it scans for the Pi relay only and reconnects by itself, backing off 1, 2, 4, then 8 s. It retries immediately when the app returns to the foreground. Direct-to-ESP32 is off by default: the ESP32 accepts one connection, so a phone holding it would lock the Pi out.

Sessions

- One reading per second is stored in SQLite: BPM and SpO₂ only when "usable", plus finger, quality and the time received.
- If the ESP32 restarts mid-drive (its sequence jumps backwards), the app closes the session and continues in a new one, so nothing collides.

Live upload (`lib/db/liveSync.ts`) runs every second. It pushes the driver, the session (with a heartbeat: `last_seen_at` and a `link_state` of streaming / sensor_lost / pi_lost), new readings and alerts. At the end of a drive it also uploads the folded archive to Storage. If the database hasn't had the v3 update, it falls back to the older columns instead of failing.

Install on an iPhone. There is no paid Apple account, so the app is sideloaded:

1. Push to the build repo, and GitHub Actions builds an unsigned `.ipa`. See `.github/workflows/ios-build.yml` : it installs, typechecks, runs the unit tests, runs `expo prebuild` and `xcodebuild` , then packages the file.
2. Download the artifact.
3. Sign and install it with **iload** and a free Apple ID. Run `apploader` on the dev laptop.
4. On the phone: **Settings** → **General** → **VPN & Device Management** → **Trust**, with Developer Mode on.

Free signing certificates expire after **7 days**. Then the app shows "no longer available"; reinstall to renew.

Supabase URL and publishable key: in CI they come from the repository secrets `EXPO_PUBLIC_SUPABASE_URL` and `EXPO_PUBLIC_SUPABASE_PUBLISHABLE_KEY` , written to `.env` at build time.

8. Cloud: Supabase

Run these in **Supabase** → **SQL Editor**, in this order. Each is safe to re-run.

1. `mobile-app/supabase/schema.sql` — creates:
 - tables `driver_profiles`, `drive_sessions`, `telemetry_events`;
 - the view `session_summaries`;
 - prototype row-level security.
2. `mobile-app/supabase/live.sql` — turns on Realtime. It also adds a bridge to the old `test_readings` table, which step 3 retires.
3. `mobile-app/supabase/v3_dashboard.sql` — adds:
 - the session heartbeat, plus finger and quality on each reading;
 - **percentile summaries**, the `driver_baselines` view and the `active_sessions` view;
 - `session_archives` and the private Storage bucket `session-archives`;
 - `drive_alerts`;
 - Realtime on all live tables;
 - and it removes the bridge.
4. `mobile-app/supabase/v4_flags.sql` — adds:
 - profile `conditions`, `medications` and `language`;
 - a `bmi` column computed by the database;
 - alert `level`, `channel` and `answer_confidence`.

The emergency contact is deliberately not stored in the cloud.
5. `mobile-app/supabase/v7_history.sql` — adds `threshold_history`, one row each time a driver's warning lines change. The website draws it on the Drivers page.
6. `mobile-app/supabase/v6_quality.sql` — adds:
 - per-second quality labels (`hr_beats`, `sqi_good`, `template_r`, `perfusion`, `skewness`);
 - calibration fields on the profile;
 - `driver_baselines` computed from clean seconds only.
7. `mobile-app/supabase/v5_delete.sql` — lets the app remove waveform files from Storage when a driver or drive is deleted. The table rows already delete and cascade without it.

All primary keys are UUIDs made on the phone, so uploads are idempotent upserts.

9. Website dashboard

`website/` is Vite with vanilla JS modules. It is deployed on Vercel at <https://smart-wheel-dashboard.vercel.app>.

- **Live**
 - Every active drive appears on its own, with no refresh.
 - When exactly one is active, a big live chart opens, with the driver's usual range drawn behind it.
 - State pills show Streaming / No finger / Sensor lost / Phone quiet, and alert banners appear here too.
 - When a drive ends, its card turns into the summary: realistic low/high, median, average, duration and usable %.
- **Drives** — history with paginated charts, alerts, and **Unfold waveform**. Unfold downloads the `.ppga` file from Storage, checks its SHA-256, and plots the red and IR waveforms with zoom.
- **Drivers** — each driver's baseline, or "learning" until there is enough data.
- **Developer feed** — the team's original `test_readings` table.
- It shows a setup banner if `v3_dashboard.sql` hasn't been run.

Local run: `cd website && npm install`, create `website/.env` with `VITE_SUPABASE_URL=...` and `VITE_SUPABASE_PUBLISHABLE_KEY=...`, then `npm run dev`.

Deploy:

```
npx vercel link
npx vercel env add VITE_SUPABASE_URL production
npx vercel env add VITE_SUPABASE_PUBLISHABLE_KEY production
npx vercel deploy --prod
```

10. Data policy, storage and backups

The proposal's policy ("Option 3"): keep processed values, not raw PPG, with restricted permissions. The app follows it by default and makes raw storage an explicit, informed choice.

MODE (SETTINGS → WHAT TO SAVE)	STORED
Vitals only (default)	BPM, SpO ₂ , finger and quality once per second. No raw samples anywhere.
Vitals + full waveform	The above, plus every raw red/IR sample, folded into an archive at the end of the drive. Use only with the driver's consent.

The folded archive ("PPGA") works like folding a long poem to keep it: small to store, and it opens back exactly when you need it. `mobile-app/lib/archive/codecs.ts` has the full byte layout.

1. **Predict each value from the previous ones.** A fixed polynomial predictor of order 0–3 is chosen per column, the same idea as FLAC's "fixed" predictors (RFC 9639). A smooth PPG waveform leaves tiny leftovers ("residuals").
2. **Store the residuals compactly:** zigzag + LEB128 variable-length integers, so small numbers take 1–2 bytes instead of 4.
3. **DEFLATE-compress the result** (LZ77 + Huffman).
4. **Verify before deleting anything.** The phone unfolds the archive and compares every sample with the original. Only then does it clear the raw copy, in one database transaction. A **SHA-256** checksum is stored and checked every time the archive is opened, on the phone or the website.

On synthetic 100 Hz PPG this folds 2 minutes from 96 KB to 22 KB, **4.4×** and lossless. Published lossless compression of biosignals using prediction plus entropy coding reports about 3–4×

 (Bannajak et al. 2023). Expect about 4× on real data. Confirm this on your own recordings.

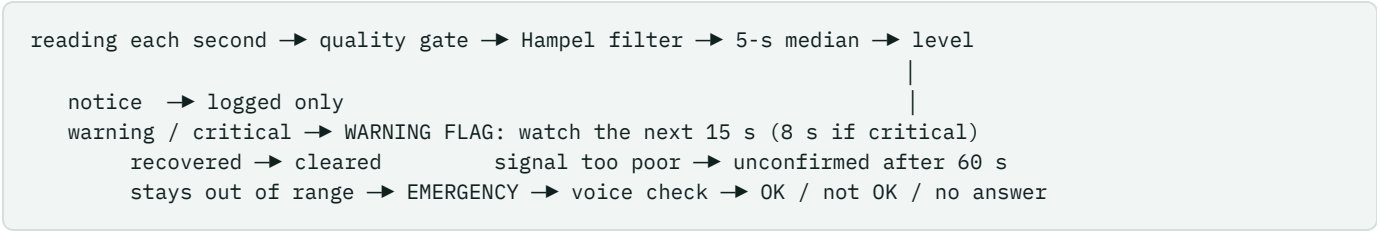
Backups (3-2-1 rule): three copies, on two kinds of storage, one off-site (CISA / US-CERT "Data Backup Options").

- **Copy 1:** the phone's database.
- **Copy 2:** Supabase (off-site). Readings, summaries and the `.ppga` archive in Storage.
- **Copy 3:** the Pi's SD card CSV of processed values.

Supabase encrypts data at rest (AES-256) and in transit (TLS).

11. Warning and emergency system

This section covers how a reading becomes a **warning flag**, how a warning becomes an **emergency**, and what the phone does next. All of it runs **on the phone**. The website only displays the results.



11.1 What is "normal" for this driver (the profile)

The profile form collects:

- age, sex, height and weight (the app computes **BMI** from these and shows it live);
- health conditions: high blood pressure, diabetes, high cholesterol, arrhythmia, sleep apnea, asthma, COPD, coronary artery disease, prior heart attack, heart failure, peripheral vascular disease, prior stroke;
- medications: beta blocker, non-DHP calcium-channel blocker, amiodarone, inhaled beta-agonist;
- the voice-check language (English or Spanish);
- an optional emergency contact, which **stays on the phone**.

Long-press a driver to edit their profile.

The expected heart rate is computed in `lib/analysis/profileModel.ts` , in three layers.

1. Population prior. This is your reference workbook's layer. It uses real-world smartphone-PPG norms from Avram et al. 2019 (66,788 people):

- It starts from the age-group mean ± SD (18–20: 81.6 ± 14.0 ... 61–70: 73.0 ± 12.7).
- It then applies the paper's multivariable coefficients:

FACTOR	ADJUSTMENT
Sex	female +2.1 BPM, male –2.1 BPM (half of the +4.28 female–male difference)
BMI	+0.21 BPM per kg/m² relative to 27.5
Conditions	diabetes +4.48, sleep apnea +3.67, COPD +2.49, hypertension +1.83, arrhythmia +1.65, asthma +1.51, high cholesterol +1.35
Medications	non-DHP calcium-channel blocker +4.11

Effects the paper found **not significant** (beta blockers, amiodarone, CAD, prior MI, CHF, PVD, stroke; $p \geq 0.05$) are recorded but not applied. Every adjustment is listed in the app under **Settings** → **Safety checks**, so the numbers can always be explained.

The starting warning line. Avram 2019 also reports the real-world **95th-percentile** heart rate by age:

- ≤ 110 BPM at 18–45;
- ≤ 100 at 45–60;
- ≤ 95 over 60.

Until the driver's own drives are known, that number, shifted by the same sex, BMI and condition adjustments, is where the **high warning** line starts. It always stays inside NEWS2's 91–111 band.

(Using the population SD instead, as an earlier version did, put every profile at the same 111 line, because people differ by ± 14 BPM. The profile then changed nothing.)

What each profile field changes

FIELD	EFFECT
Age	expected heart rate and the starting high line (110 / 100 / 95 by age group)
Sex	± 2.1 BPM on both
Height + weight $\rightarrow$ BMI	+ 0.21 BPM per kg/m ² above 27.5, – below
Diabetes, sleep apnea, COPD, high blood pressure, arrhythmia, asthma, high cholesterol	+ 1.4 to + 4.5 BPM (the paper's significant effects)
Heart disease, prior MI, heart failure, PVD, prior stroke	recorded; no significant effect in the data, so no change
COPD	oxygen judged against the driver's own baseline (§11.1 below)
Arrhythmia / AFib	irregular-pulse advisory turned off (it would only repeat what they know)
Medications	non-DHP calcium-channel blocker + 4.1 BPM; the others had no significant effect
Language	voice-check language (English / Spanish)
Name	spoken in the voice check
Emergency contact	a Call button after a "not OK" or no answer (stays on the phone)
Subject ID	labels only

Examples before any drives:

DRIVER	HIGH WARNING LINE
20-year-old man	107
55-year-old man with hypertension	100
68-year-old woman	97

DRIVER	HIGH WARNING LINE
75-year-old man	93

2. Personal baseline. The 10th, 50th and 90th percentiles of the driver's own finished drives. They gradually replace the prior, with weight $w = n / (n + 600)$: 10 minutes of good signal counts as much as the whole population prior. The SD never goes below 6 BPM.

3. Clinical bands (NEWS2). These are the same for everyone, because a dangerous value is dangerous whoever you are:

- Pulse: ≤ 40 scores 3, 41–50 scores 1, 51–90 scores 0, 91–110 scores 1, 111–130 scores 2, ≥ 131 scores 3.
- SpO₂ Scale 1: ≤ 91 scores 3, 92–93 scores 2, 94–95 scores 1.
- SpO₂ Scale 2 (confirmed hypercapnic COPD only): ≤ 83 scores 3, 84–85 scores 2, 86–87 scores 1.

COPD drivers are the exception for oxygen. People with stable COPD live at a lower normal saturation:

- In Little et al. 1999 (33 stable, normoxic or mildly hypoxic patients), awake SaO₂ was **93.9 ± 1.6 %**. That sits inside Scale 1's warning band, so Scale 1 would flag them all day long.
- The same paper defines a clinically significant desaturation as a **fall of more than 4 points from the person's own awake baseline**.

So for a driver whose profile includes COPD, oxygen is judged against **their baseline**: 93.9 % to start, then their own learned median once it is established.

- Notice: a 3-point fall.
- Warning: a fall of more than 4 points.
- Critical: ≤ 85 %, the NEWS2 Scale 2 score-2 band.

Scale 2 is not applied wholesale because it requires blood-gas-confirmed hypercapnia, which a profile cannot know.

11.2 Levels

LEVEL	HEART RATE	SPO ₂	WHAT HAPPENS
Notice	NEWS2 1 and ≥ 2.5 SD from the driver's band	94–95 % (COPD: 3-point fall)	logged only
Warning	NEWS2 2 (111–130), or NEWS2 1 and ≥ 3 SD from the band	92–93 % (COPD: fall of more than 4 points)	warning flag, then confirmation
Critical	≤ 40 or ≥ 131 (NEWS2 3)	≤ 91 % (COPD: ≤ 85 %)	warning flag, then short confirmation

For a typical 30-year-old man of normal weight with no history, the warning lines work out to **above 111** and **below 41 BPM**. An athlete whose learned baseline is around 50 BPM is not flagged at 47. Someone whose normal is 62 is warned at 91, because that is both clinically elevated and 3 SD above their normal.

11.3 Warning → emergency ("watch the next seconds")

1. **Quality gate.** A second counts only if:

- the finger is on the sensor;
- both vitals are valid;
- pulse periodicity is $\geq 50\%$.

Bad seconds count as *missing*; never as normal, never as abnormal.

2. **Hampel filter.** A single second that disagrees with its 7-second neighbourhood by more than $3 \times 1.4826 \times \text{MAD}$ is dropped. The minimum distance is 8 BPM or 3 % SpO₂.

3. **5-second median** decides the level.

4. **Confirmation window.** A warning-level median raises the **warning flag** and starts watching:

- **15 s** at warning level, **8 s** at critical level;
- it becomes an **emergency** only if $\geq 70\%$ of those seconds had an accepted reading **and** $\geq 80\%$ of them were beyond the line;
- if the median comes back (5 BPM / 1 % hysteresis), the flag clears as *recovered*;
- with too little signal for 60 s, it closes as *unconfirmed*.

Why wait? Alarm research shows short delays remove most non-actionable alarms. A 14-s delay removed 50 % and a 19-s delay 67 % of ignored or ineffective ICU alarms (Görges et al. 2009). A 6-s delay halved SpO₂ alarms (Rheineck-Leyssius & Kalkman 1998).

5. **After the check.** After "I'm OK" the same kind stays quiet for 5 minutes. If it gets worse (warning → critical) it re-arms immediately.

6. **Learning from "I'm OK".** If the driver answers OK to a **warning-level** heart-rate check, that value was evidently normal for them:

- their warning line in that direction moves to the episode's peak + 5 BPM (– 5 for low), and is saved per driver;
- limits keep this safe: the high line never passes **125**, the low line never goes below **43**, so the critical lines ($\geq 131, \leq 40$) always still ask;
- critical episodes and oxygen never adapt;
- Settings → Safety checks shows any adjustment, with **Reset**.

7. **No hand on the sensor** for 15 s during a drive raises a notice. It is not an emergency by itself: the sensor simply cannot see anything.

11.4 The voice check (phone only, free, offline)

When an emergency is confirmed, the phone vibrates, then:

1. It **speaks**, using the phone's own text-to-speech (iOS AVSpeechSynthesizer via `expo-speech`). It uses the best installed voice for the language, in this order: Premium, then Enhanced, then default, never novelty voices. Premium and Enhanced voices are free downloads: **Settings** → **Accessibility** → **Read & Speak** → **Voices** (iOS 26; called Spoken Content on iOS 17–18). They are about 100–400 MB each, on Wi-Fi.

To choose a voice yourself: **Settings** → **Safety checks** → **Voice** lists every installed voice (Premium, Enhanced, Standard). Tap one to hear it; it is used from then on. "Automatic" picks the best one.

Prompts play in iOS "voice prompt" mode, the one navigation apps use: full volume through the speaker or the car's Bluetooth, music ducked, even with the silent switch on. For example: "Luis, your heart rate has been unusually high. Are you feeling okay? Please say yes, or no."

2. It **listens for 6 s** with the phone's speech recognizer (`expo-speech-recognition` → Apple SFSpeechRecognizer):
 - **on-device when supported**, so audio never leaves the phone;
 - the recognizer is primed with the expected answers.
3. It **understands** the answer with a small local classifier (`lib/voice/intent.ts`). This is a logistic regression over word and character n-grams, about 120 KB of weights, trained by `tools/intent/train.py` on about 4,700 English and Spanish phrasings. Safety rules sit on top of the model:
 - **Urgent words** ("help", "911", "ambulance", "chest", "can't breathe", "ayuda", ...) → *not OK + urgent*, unless negated ("I don't need help").
 - **Explicit negation** ("not okay", "I don't feel well", "no estoy bien") → *not OK*.
 - **The model** decides "not OK" at ≥ 0.5 probability, but "OK" only at ≥ 0.75 . Anything in between counts as unclear. A wrong "OK" is the one mistake that matters, so "OK" needs the most evidence.
4. **Unclear or silence**: it asks once more, more simply. Still no clear answer → **no response**, which is treated like "not OK" (per the proposal).
5. **Outcome**:
 - "OK" → "I'll keep an eye on things."
 - "Not OK" or no answer → "Please pull over as soon as it is safe. If you need emergency help, call 911." It is recorded as **escalated (simulated)**, and the Drive screen shows **Call {emergency contact}** and **Call 911** buttons. The driver taps them; nothing is dialled automatically.

The wording never claims help is on the way, because in this prototype nobody is contacted.

6. **The on-screen buttons** ("I'm OK" / "I'm not OK") work at any moment and on any tab. They also cover phones where the microphone isn't allowed.

The voice check only reacts to flags. It never analyses the vitals. Transcripts are used in memory only and are **never stored or uploaded**. Only the outcome, the channel (voice or button) and the classifier's confidence are saved.

Classifier accuracy.

- Held-out set: 63 hand-written phrases, never used for training → **63/63 correct, 0 "not OK" heard as "OK"**.

- The phone's probabilities match the Python trainer's to 10^{-4} (parity test).
- Caveat: the held-out set was consulted once to add Spanish negation examples, so it is no longer perfectly blind.
- **Next step:** record real answers from team members (with road noise) and add them as a new test set.

Try it: Settings → **Try the voice check** runs the real dialogue without recording anything.

Demo — trigger a warning (Settings) plays **fabricated readings** through a fresh copy of the real engine, using this driver's thresholds, at 4× real time:

- **Scenarios:** very high heart rate (critical), high heart rate (warning, 15-s check), very low heart rate, low oxygen.
- **What you see:** about 10 normal seconds, then abnormal ones. The Drive tab shows the warning flag and the confirmation, then the real voice check asks if you're OK.
- **Nothing is kept:** nothing is saved, uploaded, or learned from.
- **Timing:** a demo takes about 5–10 real seconds to reach the voice check.

11.5 Irregular-rhythm advisory

Modelled on Apple's Irregular Rhythm Notification (Apple, *Using Apple Watch for Arrhythmia Detection*, Dec 2020). It is an **advisory shown after the drive, never an emergency and never a diagnosis**.

1. **Beats.** The Elgendi et al. 2013 detector runs on the 100 Hz infrared PPG:

- 0.5–8 Hz zero-phase band-pass, clip, square;
- moving averages over 111 ms and 667 ms, with offset $\beta = 0.02$;
- reported 99.84 % sensitivity.

2. **Segments.** Each segment is 128 beat intervals from *continuous* clean signal. Any bad second or lost packet restarts it, following Apple's rule of analysing only when there is enough signal.

3. **Irregular** means all three of the Dash et al. 2009 measures hold:

- RMSSD / mean ≥ 0.098 ;
- Shannon entropy ≥ 0.8 ;
- turning-point ratio within 0.527–0.8.

Dash reported 94.4 % sensitivity and 95.1 % specificity on MIT-BIH AF; the thresholds come from the authors' patent.

4. **Five of six** consecutive irregular segments → advisory. Two regular segments reset the count (Apple). In the Apple Heart Study sub-study, 78.9 % of notified people had AFib confirmed by ECG patch.

11.6 What gets recorded

Every episode becomes a `drive_alerts` row, which appears on the website live:

- `kind` — the type of episode;

- `level` — notice, warning or critical;
- `value` — the most extreme 5-s median;
- `threshold` — the line it crossed;
- `started_at`, `prompted_at` — when it started and when the driver was asked;
- `response` — `ok`, `not_ok`, `no_response`, `recovered` or `unconfirmed`;
- `channel` — voice or button;
- `answer_confidence`, and `escalated`.

The website shows a banner only for real warnings being confirmed and for emergencies, not for notices.

Tested scenarios (`tests/flagEngine.test.ts`, `rhythm.test.ts`, `voiceCheck.test.ts`, `intent.test.ts`):

- **Heart rate:**
 - normal driving → no flags;
 - isolated 170-BPM artifacts → rejected;
 - sustained 142 → emergency about 8 s after the flag;
 - sustained 118 → emergency after the 15-s window;
 - an 8-s burst → cleared as recovered;
 - dropouts → closed as unconfirmed.
- **Profiles and oxygen:**
 - an athlete at 47 → not flagged;
 - SpO₂ 89 → emergency, SpO₂ 93 → emergency for a typical driver;
 - a COPD driver at their normal 92–94 % → not flagged, but a drop to 88 % → emergency; with a learned baseline of 96 %, the warning line moves to 91 %.
- **After the check:**
 - the 5-min quiet period holds, and critical re-arms;
 - hands off the wheel → notice only.
- **Rhythm:** sinus rhythm is never advised; AF-like rhythm is advised after 5 of 6 segments; bad signal is never analysed.
- **Voice dialogue:** yes / unclear → no / silence×2 / urgent / Spanish / button mid-listen.

11.7 Accuracy and learning

Accuracy on a steering wheel is hard. In the only real-road study of wheel sensors we found, beat detection scored just **45–58 %** (Warnecke et al. 2023, 19 drivers), mainly because of hand position and vehicle motion. So the app works in three steps: judge every second first, learn only from good seconds, and keep checking itself against the driver.

1. Every second is quality-checked on the phone (`lib/analysis/signal.ts`). It analyses the last **10 s** of the 100 Hz waveform:

- **Beat heart rate:** the median of beat-to-beat intervals from the Elgendi detector. This is a second heart-rate estimate, independent of the ESP32.

- **Feasibility** (Orphanidou et al. 2015):
 - rate 40–180 BPM;
 - no gap over 3 s;
 - longest / shortest interval < 2.2.
- **Template match** (Orphanidou 2015): every beat is correlated with the window's average beat. It must be ≥ 0.86 (reported 91 % sensitivity, 95 % specificity).
- **Skewness** (Elgendi 2016, the best single quality index): clean PPG is positively skewed, so it must be ≥ 0 .
- **Agreement:** the ESP32's heart rate and the beat heart rate must agree within ± 5 BPM (the criterion used by Gao et al. 2025 for in-vehicle PPG).

A second is **good** only if all of these hold. Only good seconds count toward warnings, baselines and trends.

Perfusion index (pulse strength) is recorded and shown as a "weak pulse" hint, but it does not reject seconds. The published cut-offs are for clinical clip probes, and Elgendi found the perfusion index not significant.

Every quality metric is **saved with the reading** (`v6_quality.sql`), so the dataset carries its own labels for future algorithm work.

2. Learning the driver (`learnBaseline` in `lib/analysis/baseline.ts`):

- **Window:** the last **28 days** of drives, like the wearable studies (Mishra et al. 2020; Alavi et al. 2022). If there are fewer than 10 drives in it, it uses the last **10 drives** (Cacheda et al. 2026 require ≥ 10 observations).
- **Only good seconds count.**
- **Emergencies are never learned as normal:** everything from 60 s before to 120 s after an episode the driver answered "not OK" (or didn't answer) is excluded.
- **Driving data only.** Heart rate while driving runs about 11 BPM above rest (a taxi-driver study), so resting norms would mislead.
- **Personal thresholds take over gradually:** $\text{weight} = \min(1, \text{drives} / 10) \times \text{seconds} / (\text{seconds} + 600)$.
- "I'm OK" answers adjust the warning line (§11.3).

3. Trend check (`lib/analysis/trend.ts`). After each drive:

- "This week" = the median of the per-drive median heart rates over the last 7 days (at least 3 drives).
- "Usual" = the same over the 28 days before that (at least 10 drives).
- It is flagged when this week is at least **4 BPM** (Alavi 2022) and **0.5 SD** (Radin 2020) above usual. The SD is floored at 3 BPM, the typical day-to-day variation (Quer 2020).
- This is how the wearable studies spotted illness days before symptoms; the median elevation Mishra found was 7 BPM.
- It shows on the Drive screen and is logged once a week as an advisory. It is never an alarm and never a diagnosis.

4. Personal calibration (Settings → Calibrate with a reference device):

- Sit still for 60 s with a reference device on the other hand. The app records the wheel's clean seconds, and you enter the reference average.

- **Heart rate:** the difference becomes a personal correction of at most ± 10 BPM. It is **averaged over every session**, because agreement studies use about 100 paired readings (Bland). The target is within ± 10 % (the consumer standard CTA-2065) and ideally a bias of about 2–3 BPM.
- **Oxygen** is compared but **never corrected**. The FDA validates oximeters against arterial blood, and a home oximeter has its own 2–3 % error.
- SpO₂ is labelled an *estimate* everywhere. Pulse oximeters miss low oxygen about 3× more often in Black patients (Sjoding et al. 2020: 11.7 % vs 3.6 % occult hypoxaemia).

What would make it more accurate next (hardware):

- An accelerometer or gyroscope on the ESP32, to discard seconds while steering. Babusiak et al. 2021 did this with the wheel's gyroscope Z axis.
- A second sensor under the other hand (the proposal's multiple sensing points).
- Validating against a chest-strap ECG on real drives, then tuning the thresholds on the team's own labelled data.

11.8 Watching it adapt, and tuning it from the team's data

It adapts automatically, per driver, after every drive:

- the warning lines move toward the driver's own normal (28-day window, full weight after 10 drives);
- "I'm OK" answers move a warning line past a false alarm (never beyond 125 or below 43);
- COPD oxygen follows the driver's own level;
- the weekly trend compares each week with the driver's own previous four.

Simulated calm driver (30-year-old man, true driving HR about 64):

DRIVES	LEARNED FROM OWN DATA	WARN ABOVE / BELOW
0	0 %	107 / 41
5	44 %	106 / 41
10	94 %	91 / 45
20	97 %	91 / 46

What never adapts per driver, on purpose:

- the critical lines (≤ 40 / ≥ 131 BPM, oxygen ≤ 91 %), which are dangerous for anyone;
- the rules for a clean second, and the confirmation windows.

Letting these drift per driver would let a noisy sensor teach the system to trust bad data, or let a driver train it into ignoring real events.

Adaptation history. Every change is saved as a snapshot: when, why (after a drive, an "I'm OK", a profile change, a reset), how many drives it learned from, the expected heart rate and the lines themselves.

- In the app: Settings → Safety checks → *How your thresholds changed*.

- On the website: the Drivers page draws each driver's lines over time.
- Table: `threshold_history` (`v7_history.sql`).

Team tuning report (`cd mobile-app && npm run tune`; `npm run tune:demo` shows it on synthetic data). It reads everyone's labelled drives from Supabase and reports:

1. How much of each driver's data is clean.
2. Agreement between the ESP32 and the phone's beat heart rate: Bland–Altman bias and 95 % limits, and MAPE against the ± 10 % consumer target.
3. The best **weak-pulse (perfusion) cut-off for our sensor** (Youden J, AUC). The research said this must be learned from our own data.
4. Alerts in practice: false-alarm share and emergencies per 10 driving hours.
5. A **replay of every drive** through the real engine with shorter or longer confirmation windows and with the quality gate off, showing how many "not OK" episodes each setting catches and how many alarms it costs.

It only *recommends*. A person reviews the report and changes the constants in `lib/analysis`, which is how medical algorithms are tuned: on the whole dataset, with review, not by each phone rewriting its own safety rules. The report is written to `mobile-app/tools/tuning/report.md` (gitignored; drivers are numbered, not named).

On 2026-09-21 the live database held 2 drivers and 9 drives (0.1 h) — enough to prove the pipeline, not yet enough to tune.

These thresholds are prototype values built from published references. They are not clinically validated for this device. Validating them on real drives is future work.

12. Setting up a development machine (any Linux)

1. System packages

DISTRO	COMMAND
Debian / Ubuntu / Raspberry Pi OS	<code>sudo apt install git python3 python3-venv python3-pip g++ bluez curl chromium</code>
Fedora	<code>sudo dnf install git python3 python3-pip gcc-c++ bluez curl chromium</code>
Arch / Manjaro / Omarchy	<code>sudo pacman -S git python python-pip gcc bluez bluez-utils curl chromium</code>
openSUSE	<code>sudo zypper install git python3 python3-pip gcc-c++ bluez curl chromium</code>

Then enable Bluetooth: `sudo systemctl enable --now bluetooth`.

2. Node.js 22. Use a version manager so every distro gets the same version:

```
curl -fsSL https://fnm.vercel.app/install | bash      # then open a new terminal
fnm install 22 && fnm default 22
node --version                                       # v22.x or newer
```

3. The code

```
git clone <repo-url> fiu-senior && cd fiu-senior
git checkout luis/full-system
```

4. ESP32 toolchain

```
curl -fsSL https://raw.githubusercontent.com/arduino/arduino-cli/master/install.sh | BINDIR=~/.local/bin
sh
~/.local/bin/arduino-cli config init
~/.local/bin/arduino-cli config add board_manager.additional_urls https://espressif.github.io/arduino-
esp32/package_esp32_index.json
~/.local/bin/arduino-cli core update-index
~/.local/bin/arduino-cli core install esp32:esp32@3.3.12
~/.local/bin/arduino-cli lib install "SparkFun MAX3010x Pulse and Proximity Sensor Library@1.1.2"
```

Serial-port access for the ESP32's CP210x chip, needed once per machine. Replug the ESP32 afterwards:

```
echo 'SUBSYSTEM=="tty", ATTRS{idVendor}=="10c4", ATTRS{idProduct}=="ea60", TAG+="uaccess"' | sudo tee
/etc/udev/rules.d/70-esp32-cp210x.rules
sudo udevadm control --reload-rules && sudo udevadm trigger
```

5. Python tools (laptop viewer, esptool)

```
python3 -m venv ~/ppg-venv
~/ppg-venv/bin/pip install bleak==3.0.2 dbus-fast pyqtgraph PySide6 pyserial esptool rich numpy
```

6. Retraining the voice-answer model (only needed if you change the phrases in `mobile-app/tools/intent/phrases.py`). It needs Python 3 with numpy:

```
cd mobile-app && python3 tools/intent/train.py && python3 tools/intent/check.py && npm test
```

It prints the held-out accuracy and the number of critical errors ("not OK" heard as "OK"). That number must stay 0.

7. Phone app and website dependencies

```
cd mobile-app && npm ci && npm test && cd ..
cd website && npm ci && cd ..
```

13. Everyday commands

TASK	COMMAND
Flash the ESP32 (USB)	<code>system/esp32/flash_esp32.sh</code> (optional port: <code>/dev/ttyUSB0</code>)
Watch the ESP32	<code>~/local/bin/arduino-cli monitor -p /dev/ttyUSB0 -c baudrate=115200</code> (opening the port resets the board)
Install/refresh the Pi	copy <code>system/pi/</code> to the Pi (USB stick or <code>scp -r system/pi pi:</code>), then on the Pi: <code>sudo bash setup_pi.sh</code>
Pi service status / logs	<code>systemctl status ppg-relay</code> · <code>journalctl -u ppg-relay -f</code>
Pi dashboard	<code>ppg-monitor</code> (starts automatically at boot)
Pi Wi-Fi off / on	<code>sudo nmcli radio wifi off</code> · <code>sudo nmcli radio wifi on</code>
Laptop viewer	<code>system/laptop/run_viewer.sh</code> (via the Pi) · <code>--direct</code> (ESP32) · <code>--demo</code> (no hardware)
Python tests	<code>cd system && python3 -m unittest discover -s tests -v</code>
App tests + typecheck	<code>cd mobile-app && npm test && npm run typecheck</code>
Build the iPhone app	push the branch to the build repo → Actions → download <code>SmartWheelApp-unsigned-ipa</code>
Install on iPhone	<code>apploader</code> (opens iloader with the newest <code>.ipa</code> path on the clipboard)
Team tuning report (from Supabase)	<code>cd mobile-app && npm run tune</code> (or <code>npm run tune:demo</code>)
Retrain / check the voice-answer model	<code>cd mobile-app && python3 tools/intent/train.py</code> · <code>python3 tools/intent/check.py</code>
Website locally	<code>cd website && npm run dev</code> → <code>http://localhost:5173</code>
Deploy website	<code>cd website && npx vercel deploy --prod</code>

14. Tests

SUITE	WHAT IT PROVES
<code>system/tests/test_protocol.py</code> (20 tests)	CRC check value; v2 and v3 round-trip; every single-bit flip is detected ; deframing at any chunk size; resync after garbage and false headers; the Python decoder reads packets from the compiled firmware encoder and the Python encoder reproduces them byte for byte; the shared modules in <code>pi/</code> and <code>laptop/</code> match the master copy
<code>system/tests/test_vitals_host.cpp</code>	the heart-rate/SpO ₂ estimator on 45 synthetic signals with known answers
<code>mobile-app/tests/protocol.test.ts</code>	the phone decodes packets from the compiled firmware encoder; bit flips; mixed v2/v3 streams at Bluetooth chunk sizes; the false-header look-ahead
<code>mobile-app/tests/codec.test.ts</code>	folding is lossless, handles edge cases, rejects corruption; reports the compression ratio
<code>mobile-app/tests/flagEngine.test.ts</code>	profile prior = Avram coefficients; NEWS2 bands; 11 driving scenarios: artifacts rejected, critical and warning confirmation windows, recovery, poor signal, athlete baseline, SpO ₂ scales, cooldown and re-arm, no contact
<code>mobile-app/tests/rhythm.test.ts</code>	beat detection on a synthetic PPG; sinus vs AF-like intervals; 5-of-6 advisory; bad signal never analysed
<code>mobile-app/tests/intent.test.ts</code>	the phone's hashing and probabilities equal the Python trainer's; held-out phrases with zero critical errors; safety rules (urgent words, negation, "no, I'm fine", Spanish)
<code>mobile-app/tests/voiceCheck.test.ts</code>	the spoken dialogue: yes, unclear then no, silence twice, urgent, Spanish, a button press mid-listen
<code>mobile-app/tests/stats.test.ts</code>	percentiles match Postgres; learning (28-day window, episodes excluded, unclear seconds ignored); weekly trend; calibration limits
<code>mobile-app/tests/signals.test.ts</code>	clean pulse accepted; motion, noise and estimator disagreement rejected; 10-s window reset on gaps
<code>mobile-app/tests/safetyController.test.ts</code>	demo: fabricated readings → warning → emergency → voice check, with nothing saved or learned
<code>mobile-app/tests/shared-sync.test.ts</code>	the website's copy of the codec is byte-identical

SUITE	WHAT IT PROVES
SQL	<code>v3_dashboard.sql</code> was run twice on Postgres 17 (PGLite) with the Supabase roles, bucket and publication stubbed

CI runs the app typecheck and unit tests on every build.

15. Measured results

MEASUREMENT	RESULT
Acquisition window	990–1003 ms per 100 samples, 0 FIFO overflows
Packet	v3, 100 samples, 472 bytes, 1 per second
Laptop link (bench)	0 CRC errors, packets every 1.004 s
Heart-rate estimator vs Maxim (same pulse; reference 79–80 BPM)	estimator median 65, σ 3.3 · Maxim median 115
Finger threshold evidence	finger $\approx$ 242 000 · object $\approx$ 50 400 · air $\approx$ 1 100 (IR counts)
Pi ↔ ESP32 with Pi Wi-Fi on (2.4 GHz)	repeated 0x3E failures; the Pi heard the ESP32 at –80 to –93 dBm with a laptop in between
Pi ↔ ESP32 with Pi Wi-Fi off	connected and steady
Relay before/after stale-disconnect fix (laptop, same ESP32)	dropped after 3–4 s every time (0x13) → held 145 s, until the test ended
Archive fold (synthetic, 2 min @ 100 Hz)	96 KB → 22 KB (4.4×), lossless

16. Troubleshooting

SYMPTOM	LIKELY CAUSE → FIX
Pi dashboard stuck on SCANNING / connecting then dropping	Wi-Fi/Bluetooth sharing on the Pi 5 or distance → turn Pi Wi-Fi off, keep the ESP32 within 1–2 m with nothing metal between
ESP32 link drops every ~3 s, ESP32 shows <code>lastdisc=0x13</code>	old relay acting on a stale disconnect event → deploy the current <code>system/pi/ppg_relay.py</code> and <code>sudo systemctl restart ppg-relay</code>
App says "Wheel quiet" while the Pi shows ESP32 and phone connected	old relay: its own "subscribed" flag fell out of step with BlueZ (relay restarted with the phone connected, or another client left) and it stopped sending → deploy the current relay, which always hands frames to BlueZ; meanwhile force-quit and reopen the app
Pi dashboard says no phone while the app streams	relay restarted while the phone stayed connected (BlueZ keeps the link) → current relay picks up already-connected clients at start
Relay logs <code>ESP32 link error: TimeoutError()</code> on every attempt	Pi Wi-Fi on (shared radio) → Wi-Fi off; or a dead BlueZ link (<code>bluetoothctl info 70:4B:CA:6F:36:86</code> says Connected: yes while the ESP32 says link=down) → the current relay clears it itself; by hand: <code>bluetoothctl disconnect 70:4B:CA:6F:36:86</code>
Pi Wi-Fi glitches when Bluetooth is on	same shared radio → use Ethernet for development, or Wi-Fi off
"phone relay unavailable ... is Bluetooth on?" in the Pi log	Bluetooth was off → switch it on; the relay retries every 5 s by itself
ESP32 serial shows "MAX30102 not found"	wiring. Read the diagnostic line: no pull-ups = power/GND; pull-ups but no ACK = SDA/SCL swapped or wrong pins
ESP32 boot loop, "Checksum failed"	flashed at 80 MHz QIO → use <code>flash_esp32.sh</code> (40 MHz DIO)
App says "no longer available"	the 7-day free signing expired → reinstall with <code>uploader</code>
iPhone: "Untrusted Developer"	Settings → General → VPN & Device Management → trust your Apple ID
Website shows "Database needs the v3 update"	run <code>v3_dashboard.sql</code>
No finger / "Place your hand on the sensor"	the IR level is below 100 000; press a fingertip flat on the sensor
Packet line shows v2 · 8 samples	the ESP32 has old firmware → reflash
The voice check doesn't listen ("buttons only")	the microphone or speech permission was denied → iPhone Settings → Smart Wheel → allow Microphone and Speech Recognition
The voice sounds robotic	download an "Enhanced" voice: iPhone Settings → Accessibility → Spoken Content → Voices
Too many warnings for one driver	check their profile (age, conditions) in Settings → Safety checks; after about 10 minutes of their own drives the thresholds follow their personal normal

17. Security status and before-real-data checklist

The current state is a **closed prototype**:

- Supabase row-level security is **permissive**. Anyone holding the publishable key, which ships inside the app and the website, can read and write every table and archive.
- The website is public at its Vercel URL.
- Bluetooth pairing uses "Just Works", with no passkey.

Before collecting real subject data:

1. Add Supabase Auth and replace the `prototype_all` policies with per-user policies (e.g. `auth.uid() = owner_id`), including the Storage bucket.
2. Put the website behind login (Supabase Auth), or Vercel Deployment Protection.
3. Bluetooth: bond the phone to the Pi, and require encryption on the relay's characteristics.
4. Consider client-side encryption of `.ppga` archives, with the key in the phone's Keychain.
5. Collect written consent, especially for "Vitals + full waveform".

18. How this maps to the proposal

PROPOSAL ITEM	STATUS
PPG sensing on the wheel, ESP32 preprocessing, BPM/SpO ₂	✓ MAX30102 + ESP32, estimator in §4
Sensor polling 25–250 Hz	✓ 100 Hz, all samples transmitted
Baseline acquisition 6–8 s	✓ 8 s analysis window
Wireless ESP32 → Pi (BLE)	✓ with CRC and end-to-end verification
Filter environmental and electrical noise	✓ decimation, baseline removal, smoothing, finger gate, plausibility, 10-s median for alerts, percentile summaries
User profiles and baselines	✓ driver picker, per-driver baseline (phone and cloud)
Real-time display	✓ phone app, Pi dashboard, live website
Store processed, time-stamped data; raw not primary	✓ vitals-only default; raw is opt-in and folded
Encryption and restricted permissions	• TLS + AES-256 at rest (Supabase), owner-only files on the Pi; per-user access control is on the checklist in §17
Baseline comparison per user profile	✓ profile prior from age, sex, BMI, conditions and medications, blended with the learned personal baseline (§11.1)
Alert → driver response → escalation	✓ warning flag → confirmation window → spoken "Are you feeling okay?" with on-device speech understanding → escalation simulated (as specified)
Haptic / non-visual alert	✓ vibration plus a spoken check (eyes stay on the road); a haptic motor in the wheel is future hardware
Hands-off-wheel alert (survey request)	✓ no-contact notice after 15 s
Website always available	✓ Vercel
Pi touchscreen UI	• the terminal dashboard runs on the Pi's screen; a touch GUI is future work
Multiple PPG sensing points	• one sensor today; the protocol's per-packet sample count and flags leave room to add channels in a v4
Machine-learning assessment	• per-driver learning and team-level tuning with review are in place (§11.7–11.8); a trained model awaits enough labelled drives. The 100 Hz archives and per-second quality labels are that training data

19. Scaling the wheel: 12-16 sensor pads

The bench prototype has two MAX30102 modules behind one TCA9548A multiplexer (Andrew's wiring drawing, revision A, 2026-09-22). The wheel build puts 12-16 pads around the rim so a hand finds one wherever it grips. This section is the requirement list for that step; none of it is built yet.

Only one pad streams at a time. The firmware scans the pads for skin contact, streams the one with the best signal, keeps a second warmed up as the standby, and hands over when the first hand lets go. Everything downstream -- the packet, the relay, the app, the database, the website -- keeps working unchanged, because it still receives one 1 s packet per second.

That choice is what makes 16 pads affordable. Streaming them all would be about 7.6 kB/s instead of 472 B/s and would force a new packet format, a new database schema and a new upload path.

Electrical

ITEM	BENCH NOW	12-16 PADS
Multiplexers	one TCA9548A at 0x70	two (8 channels each). Addresses are set by pins A0-A2, so 0x70 and 0x71 ; up to 8 are possible
I²C pull-ups	the modules' own	one pair per active branch. Wheel-length wiring adds capacitance: check the SCL/SDA rise time on a scope before trusting the bus
5 V supply	2 modules	measure one module's current at the LED brightness the firmware sets, multiply by the pad count, add margin, then size the converter. Do not take a number from a web page: the draw depends on brightness and pulse width
Harness	jumpers	a ring harness, with a plan for crossing the steering column (slip ring or a wire loop with enough slack for full lock-to-lock)
Placement	two fixed points	the pads must sit where hands actually rest; see the contact rate measured per pad (below)

Firmware

- **Scan.** Switch the multiplexer to each pad in turn and read its IR DC level. Contact is the same test the firmware already uses (§3): the IR mean above the finger threshold, which on our module is well above the ~50 000 counts a resting object gives.
- **Choose.** Among pads with contact, stream the one with the best signal quality, not simply the first found. The phone already computes a quality score per second (§4); the firmware's cheap equivalent is its periodicity `quality` value.
- **Standby.** Keep a second pad selected and sampling so a handover does not restart the 4 s estimator window from nothing.
- **Handover.** When the streaming pad loses contact, switch to the standby and promote the next candidate. Mark the packet so the receiver knows the source changed.
- **Bus timing.** Do not switch the multiplexer for every sample. Each module buffers samples in its own FIFO (32 deep), so read a burst per pad. A switch-per-sample schedule wastes most of the I²C bandwidth on overhead.

Packet

No new version is needed. The v3 header's `flags` byte has four unused bits (`0x10 - 0x80`), which hold a pad index 0-15. Packets from the current one-sensor firmware carry zeros there, so old captures stay valid and decoders that ignore the field keep working. Whatever is decided, change it in `system/common/ppg_protocol.py` and the copies, `ppg_frame.h` , and `mobile-app/lib/ble/protocol.ts` together -- the tests compare them byte-for-byte (§14).

What each pad count buys

Report contact rate per pad from a real drive (the app already stores a `finger` flag per second, §8), then add pads where hands actually rest. 16 pads with poor placement are worse than 8 placed from measured data.

Drawing it

The prototype drawing is a picture, not an electrical design: no netlist, no rule checks, no parts list. For 12-16 repeated pads, redraw it in **KiCad** (free, and what reviewers expect): one sheet reused per pad, electrical rule checks, and a parts list generated from the schematic. A drawing tool (draw.io, Figma) is fine for the report figure, but keep the electrical truth in KiCad.

20. References

- Analog Devices / Maxim, *MAX30102 datasheet*, 19-7740 Rev 1 (2018):
 - SpO₂-mode sample rates vs pulse width (Table 11);
 - registers 0x08 FIFO_CONFIG and 0x0A SPO2_CONFIG;
 - 18-bit ADC and a 32-sample FIFO.

<https://www.analog.com/media/en/technical-documentation/data-sheets/MAX30102.pdf>
- Choi A., Shin H., "Photoplethysmography sampling frequency: pilot assessment of how low can we go to analyze pulse rate variability with reliability?", *Physiol. Meas.* 38(3):586–600, 2017.
<https://pubmed.ncbi.nlm.nih.gov/28169836/>
- Béres S., Holczér L., Hejje L., "On the Minimal Adequate Sampling Frequency of the Photoplethysmogram for Pulse Rate Monitoring and Heart Rate Variability Analysis in Mobile and Wearable Technology", *Meas. Sci. Rev.* 19(5):232–240, 2019. <https://sciencedirect.com/article/10.2478/msr-2019-0030>
- Bannajak K. et al., "Signal Acquisition-Independent Lossless Electrocardiogram Compression Using Adaptive Linear Prediction", *IJERPH* 20(3):2753, 2023. <https://pubmed.ncbi.nlm.nih.gov/36768118/>
- IETF RFC 9639, *Free Lossless Audio Codec (FLAC)*, 2024 (fixed predictors + Rice coding). <https://www.rfc-editor.org/rfc/rfc9639.txt>
- Ruggiero P., Heckathorn M. A., *Data Backup Options*, US-CERT/CISA, 2012 (the 3-2-1 rule).
https://www.cisa.gov/sites/default/files/publications/data_backup_options.pdf
- Elgendi M., "Optimal Signal Quality Index for Photoplethysmogram Signals", *Bioengineering* 3(4):21, 2016.
<https://pubmed.ncbi.nlm.nih.gov/28952584/>
- Bent B. et al., "Investigating sources of inaccuracy in wearable optical heart rate sensors", *npj Digit. Med.* 3:18, 2020. <https://pubmed.ncbi.nlm.nih.gov/32047863/>
- Royal College of Physicians, *National Early Warning Score (NEWS) 2*, 2017 (pulse and SpO₂ scoring bands; Scales 1 and 2). https://www.rcp.ac.uk/media/alxev00t/news2-chart-1_the-news-scoring-system_0_0.pdf
- Avram R. et al., "Real-world heart rate norms in the Health eHeart study", *npj Digital Medicine* 2:58, 2019 (age strata, the sex/BMI/condition coefficients, 95th percentiles). <https://doi.org/10.1038/s41746-019-0134-9>
- Team 18 reference-profiles workbook (`ppg_reference_profiles_research_backed.xlsx` , derived from Avram 2019).
- Apple Inc., *Using Apple Watch for Arrhythmia Detection*, December 2020 (Irregular Rhythm Notification: tachograms, 5-of-6 confirmation, Apple Heart Study sub-study PPV).
- Görges M., Markewitz B. A., Westenskow D. R., "Improving alarm performance in the medical intensive care unit using delays and clinical context", *Anesth Analg* 108(5):1546–52, 2009.
<https://pubmed.ncbi.nlm.nih.gov/19372334/>

- Rheineck-Leyssius A. T., Kalkman C. J., *J Clin Monit Comput* 14(3):151–6, 1998 (SpO₂ alarm delays and averaging). <https://pubmed.ncbi.nlm.nih.gov/9676861/>
- Little S. A., Elkholy M. M., Chalmers G. W., Farouk A., Patel K. R., Thomson N. C., "Predictors of nocturnal oxygen desaturation in patients with COPD", *Respiratory Medicine* 93:202–207, 1999 (awake SaO₂ 93.9 ± 1.6 % in stable COPD; significant desaturation = a fall of more than 4 % from the awake baseline). [https://doi.org/10.1016/S0954-6111\(99\)90009-4](https://doi.org/10.1016/S0954-6111(99)90009-4)
- Orphanidou C. et al., "Signal-quality indices for the electrocardiogram and photoplethysmogram: derivation and applications to wireless monitoring", *IEEE J Biomed Health Inform* 19(3):832–838, 2015. https://www.robots.ox.ac.uk/~davidc/pubs/jbhi_sqi2015.pdf
- Gao et al., in-vehicle PPG signal quality, *Sensors* 2025 (±5 BPM agreement criterion). <https://pmc.ncbi.nlm.nih.gov/articles/PMC12736534/>
- Warnecke J. M., Lasenby J., Deserno T. M., *Sci Rep* 2023 (steering-wheel PPG/ECG on real roads). <https://pmc.ncbi.nlm.nih.gov/articles/PMC10682004/>
- Babusiak B. et al., *Sensors* 2021 (MAX30102 on a steering wheel, gyroscope artefact rejection). <https://pmc.ncbi.nlm.nih.gov/articles/PMC8399225/>
- Radin J. M. et al., *Lancet Digital Health* 2020 (weekly RHR > 0.5 SD above a personal baseline).
- Mishra T. et al., "Pre-symptomatic detection of COVID-19 from smartwatch data", *Nat Biomed Eng* 2020 (28-day baselines). <https://pmc.ncbi.nlm.nih.gov/articles/PMC9020268/>
- Alavi A. et al., *Nat Med* 2022 (NightSignal, +4 BPM). <https://pmc.ncbi.nlm.nih.gov/articles/PMC8240687/>
- Quer G. et al., *PLOS One* 2020 (92,457 adults; within-person daily RHR SD ≈ 3 BPM). <https://journals.plos.org/plosone/article?id=10.1371%2Fjournal.pone.0227709>
- Cacheda F. et al., *Digital Health* 2026 (≥ 10 observations for a personal baseline). <https://pmc.ncbi.nlm.nih.gov/articles/PMC13487130/>
- Heart rate while driving vs rest (taxi drivers), *Ann Occup Environ Med* 2016. <https://pmc.ncbi.nlm.nih.gov/articles/PMC5054562/>
- Sjoding M. W. et al., "Racial bias in pulse oximetry measurement", *NEJM* 383:2477, 2020. <https://pmc.ncbi.nlm.nih.gov/articles/PMC7808260>
- Bland J. M., sample size for agreement studies. <https://www-users.york.ac.uk/~mb55/meas/sizemeth.htm>
- Nelson B. W., Allen N. B., *JMIR mHealth* 2019 (±10 % MAPE standard). <https://pmc.ncbi.nlm.nih.gov/articles/PMC6431828/>
- Pearson R. K. et al., "Generalized Hampel Filters", *EURASIP J Adv Signal Process* 2016:87. <https://doi.org/10.1186/s13634-016-0383-6>
- Elgendi M. et al., "Systolic peak detection in acceleration photoplethysmograms measured from emergency responders in tropical conditions", *PLoS ONE* 8(10):e76585, 2013. <https://pmc.ncbi.nlm.nih.gov/articles/PMC3805543/>

- Dash S., Chon K. H., Lu S., Raeder E. A., "Automatic real time detection of atrial fibrillation", *Ann Biomed Eng* 37(9):1701–9, 2009; thresholds from US patent 8,417,326 B2.
<https://patents.google.com/patent/US8417326B2/en>
- McManus D. D. et al., *Heart Rhythm* 10(3):315–9, 2013 (smartphone-PPG AF detection with RMSSD/mean + ShE). <https://pubmed.ncbi.nlm.nih.gov/23220686/>
- Witting M. D., Scharf S. M., "Diagnostic room-air pulse oximetry: effects of smoking, race, and sex", *Am J Emerg Med* 26(2):131–6, 2008 (awake adults: 5.7 % below 97 %). <https://doi.org/10.1016/j.ajem.2007.04.002>
- FDA, *Pulse Oximeters – Premarket Notification Submissions*, 2013 (reflectance accuracy Arms ≤ 3.5 %). <https://www.fda.gov/media/72470/download>
- Apple, SFSpeechRecognitionRequest requiresOnDeviceRecognition .
<https://developer.apple.com/documentation/speech/sfspeechrecognitionrequest/requiresondevicerecognition>
- expo-speech-recognition (MIT). <https://github.com/jamsch/expo-speech-recognition> · Expo Speech.
<https://docs.expo.dev/versions/latest/sdk/speech/>
- Vosk offline speech recognition (Apache-2.0), a fully open-source alternative recognizer.
<https://alphacephei.com/vosk/models>
- Supabase:
 - Security (AES-256 at rest, TLS): <https://supabase.com/security>
 - Realtime Postgres Changes: <https://supabase.com/docs/guides/realtime/postgres-changes>
- Apple, *Accessory Design Guidelines*, R30, §58 (Bluetooth LE connection parameters).
<https://developer.apple.com/accessories/Accessory-Design-Guidelines.pdf>
- Bluetooth SIG, *Core Specification 6.0* (LE data length up to 251 octets; attribute values up to 512 octets).
- Infineon, *CYW43455 datasheet* §5.9 (shared-antenna Wi-Fi/Bluetooth coexistence).
- Raspberry Pi, "Introducing Raspberry Pi 5" (the CYW43455 combo chip).
<https://www.raspberrypi.com/news/introducing-raspberry-pi-5/>

Three claims are engineering reasoning rather than cited findings:

- The combination rules (for example "warning = NEWS2 1 **and** ≥ 3 SD from the personal band", the 15-s / 8-s windows, the 70 % / 80 % confirmation fractions) are our design built on the sources above. They have not been validated on this device.
- "Percentiles instead of min/max for summaries" follows from the artifact literature above but is not taken from a specific paper.
- "Wi-Fi on the Pi degrades our BLE link" is based on our own bench measurements (§15), not on a vendor statement.